

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptron's, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS:15

Total Marks:25

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I Nirmal Kumar / 192412003 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Short Answer Test

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence. (5 Marks)

Answer

A perceptron should always converge if the training data is **linearly separable**. If it does not converge after many iterations, the following issues may be responsible.

Possible Causes

1. Improper Learning Rate

- A learning rate that is too high may cause the weights to oscillate without reaching the correct solution.
- A very small learning rate slows down convergence.

2. Incorrect Weight Initialization

- Initializing all weights with unsuitable values can delay convergence.
- Small random weight initialization is generally preferred.

3. Bias Handling Errors

- If the bias term is omitted or updated incorrectly, the decision boundary may not shift properly.
- This can prevent the perceptron from correctly classifying the data.

4. Incorrect Update Rule

- The perceptron learning rule must update both the weights and the bias whenever a sample is misclassified.
- Any mistake in the implementation of the update equation prevents convergence.

Debugging Strategies

- Verify that the dataset is linearly separable.
- Print the weights and bias after each training epoch.
- Check whether the prediction and update equations are implemented correctly.
- Monitor the number of classification errors in every epoch.
- Test the implementation using a simple dataset such as the AND gate.

Optimization Techniques

- Choose a suitable learning rate (e.g., 0.1).
- Initialize weights with small random values.
- Shuffle the training data before each epoch.
- Normalize input features to improve training stability.

Conclusion

By selecting an appropriate learning rate, correctly updating the weights and bias, properly initializing parameters, and verifying the implementation, the perceptron converges efficiently on linearly separable datasets with stable and faster learning.

Python Program

```
import numpy as np

# AND Gate Dataset (Linearly Separable)
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([0, 0, 0, 1])

# Initialize weights and bias
weights = np.random.rand(2)
bias = np.random.rand()

learning_rate = 0.1
epochs = 20

# Perceptron Training
for epoch in range(epochs):
    error_count = 0

    for i in range(len(X)):
        net = np.dot(X[i], weights) + bias
        prediction = 1 if net >= 0 else 0

        error = y[i] - prediction

        # Update weights and bias
        if error != 0:
            weights = weights + learning_rate * error * X[i]
            bias = bias + learning_rate * error
            error_count += 1

    print(f"Epoch {epoch + 1}: Errors = {error_count}")

# Stop if perfectly classified
if error_count == 0:
    print("Training Converged")
```

```
        break

# Display final parameters
print("\nFinal Weights:", weights)
print("Final Bias:", bias)

# Test the model
print("\nPredictions:")
for sample in X:
    net = np.dot(sample, weights) + bias
    prediction = 1 if net >= 0 else 0
    print(sample, "->", prediction)
```

```
Epoch 1: Errors = 3
Epoch 2: Errors = 3
Epoch 3: Errors = 3
Epoch 4: Errors = 2
Epoch 5: Errors = 2
Epoch 6: Errors = 1
Epoch 7: Errors = 0
Training Converged
```

```
Final Weights: [0.260146    0.38151473]
Final Bias: -0.5271723594863865
```

```
Predictions:
[0 0] -> 0
[0 1] -> 0
[1 0] -> 0
[1 1] -> 1
```

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance.

(5 Marks)

Answer

While training a **Multilayer Perceptron (MLP)** using the **Backpropagation algorithm**, the network may converge slowly or get stuck in local minima. This reduces the model's accuracy and training efficiency.

Possible Causes

1. Vanishing Gradients

- In deep neural networks, gradients become very small during backpropagation.
- As a result, the weights of earlier layers are updated very slowly, leading to slow convergence.

2. Inappropriate Activation Functions

- Using activation functions such as **Sigmoid** or **Tanh** in deep networks may cause gradient saturation.
- This slows learning and increases the chance of getting stuck in local minima.

3. Poor Weight Initialization

- Initializing all weights with the same value or with very large values can prevent effective learning.
- Proper initialization helps gradients flow efficiently.

4. Suboptimal Learning Rate

- A very high learning rate causes unstable updates and overshoots the minimum.
- A very low learning rate makes training extremely slow.

Debugging Approaches

1. Monitor the training and validation loss after each epoch.
2. Check whether gradients are becoming very small (vanishing gradients).
3. Visualize the learning curve to detect slow convergence.
4. Verify that weights are updated correctly after each iteration.
5. Test different activation functions and compare their performance.
6. Experiment with different learning rates and observe convergence speed.

Optimization Techniques

- Use **ReLU** activation instead of Sigmoid for hidden layers.
- Apply **He** or **Xavier weight initialization**.
- Normalize input data using **StandardScaler**.
- Use adaptive optimizers such as **Adam**, **RMSprop**, or **Adagrad**.

Conclusion

Slow convergence in an MLP can be caused by vanishing gradients, poor activation functions, improper weight initialization, or an unsuitable learning rate. By debugging the training process and applying optimization techniques such as ReLU activation, Adam optimizer, normalization, and proper initialization, the network trains faster and achieves better accuracy.

Python Program

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Load dataset
digits = load_digits()
X = digits.data
y = digits.target

# Normalize the input data
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create Multilayer Perceptron model
mlp = MLPClassifier(
    hidden_layer_sizes=(100,),
    activation='relu',
    solver='adam',
    learning_rate_init=0.001,
    max_iter=300,
    random_state=42
)

# Train the model
mlp.fit(X_train, y_train)

# Predict
y_pred = mlp.predict(X_test)
```

```
# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)
```

Accuracy: 0.9805555555555555

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed. (5 Marks)

Answer

A Genetic Algorithm (GA) used for Hypothesis Space Search may produce inconsistent or suboptimal solutions due to errors in chromosome representation, fitness evaluation, or genetic operations. Proper debugging and optimization improve the quality of solutions and convergence speed.

Possible Causes

1. Improper Chromosome Encoding

- If chromosomes do not correctly represent the problem, the algorithm cannot search the solution space effectively.
- Incorrect encoding may generate invalid or meaningless solutions.

2. Incorrect Fitness Function

- A poorly designed fitness function does not accurately measure solution quality.
- This may cause low-quality chromosomes to be selected for reproduction.

3. Premature Convergence

- The population becomes too similar after a few generations.
- The algorithm gets trapped in a local optimum and stops exploring better solutions.

4. Lack of Population Diversity

- Very low mutation rates or excessive selection pressure reduce diversity.
- As a result, the algorithm repeatedly produces similar solutions.

Debugging Approaches

1. Verify that chromosome encoding correctly represents all possible solutions.
2. Test the fitness function with known inputs and expected outputs.
3. Monitor the best and average fitness value in every generation.
4. Check mutation and crossover operations for implementation errors.
5. Observe whether the population diversity decreases too quickly.
6. Repeat the algorithm with different random seeds and compare the results.

Optimization Techniques

- Increase the mutation rate slightly to maintain diversity.
- Adjust selection pressure using Tournament Selection or Roulette Wheel Selection.
- Apply Elitism to preserve the best individuals in every generation.
- Increase the population size for better exploration.
- Use adaptive mutation and crossover probabilities.
- Terminate the algorithm when the fitness value no longer improves.

Conclusion

The performance of a Genetic Algorithm depends on proper chromosome encoding, an effective fitness function, sufficient population diversity, and balanced genetic operators. By debugging each component and applying optimization techniques such as mutation tuning, selection pressure adjustment, and elitism, the algorithm produces more accurate and consistent solutions with faster convergence.

Python Program

```
import random

# Fitness function: maximize the number of 1s
def fitness(chromosome):
    return sum(chromosome)

# Create initial population
population_size = 10
chromosome_length = 8

population = [
    [random.randint(0, 1) for _ in range(chromosome_length)]
    for _ in range(population_size)
]

generations = 20
mutation_rate = 0.1

for generation in range(generations):

    # Sort population by fitness (Elitism)
    population = sorted(population, key=fitness, reverse=True)

    best = population[0]
    print("Generation", generation + 1,
          "Best Fitness =", fitness(best),
          "Best Chromosome =", best)

    # Stop if optimal solution is found
    if fitness(best) == chromosome_length:
```

```

        break

new_population = population[:2] # Elitism

while len(new_population) < population_size:

    parent1 = random.choice(population[:5])
    parent2 = random.choice(population[:5])

    # Single-point crossover
    point = random.randint(1, chromosome_length - 1)
    child = parent1[:point] + parent2[point:]

    # Mutation
    for i in range(chromosome_length):
        if random.random() < mutation_rate:
            child[i] = 1 - child[i]

    new_population.append(child)

population = new_population

```

```

Generation 1 Best Fitness = 7 Best Chromosome = [1, 1, 1, 1, 1, 1, 1, 0]
Generation 2 Best Fitness = 7 Best Chromosome = [1, 1, 1, 1, 1, 1, 1, 0]
Generation 3 Best Fitness = 7 Best Chromosome = [1, 1, 1, 1, 1, 1, 1, 0]
Generation 4 Best Fitness = 8 Best Chromosome = [1, 1, 1, 1, 1, 1, 1, 1]

```

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability.

(5 Marks)

Answer

A **Deep Neural Network (DNN)** combined with **Genetic Programming (GP)** can solve complex prediction problems effectively. However, the model may suffer from **overfitting** and **high computational cost** due to excessive model complexity, over-parameterization, and insufficient training data. Proper debugging and optimization techniques are required to improve accuracy and efficiency.

Possible Causes

1. Model Complexity

- A very deep network with many hidden layers and neurons increases the number of parameters.
- This results in high memory usage and longer training time.

2. Over-Parameterization

- Too many trainable parameters cause the model to memorize the training data instead of learning general patterns.
- This leads to overfitting.

3. Insufficient Training Data

- A small dataset cannot represent all possible input patterns.
- The model performs well on training data but poorly on unseen test data.

4. High Computational Cost

- Deep neural networks and genetic programming require repeated evaluation of many candidate solutions.
- This increases execution time and computational resources.

Debugging Approaches

1. Compare training accuracy and validation accuracy to detect overfitting.
2. Monitor training and validation loss after each epoch.
3. Count the number of trainable parameters in the model.
4. Measure training time and memory usage.
5. Evaluate the model using cross-validation.
6. Check whether genetic programming generates unnecessarily complex solutions.

Optimization Techniques

- Apply **L2 Regularization** to reduce overfitting.
- Use **Dropout** layers to prevent neuron co-adaptation.
- Perform **Model Pruning** to remove unnecessary weights and neurons.
- Use **Early Stopping** to stop training when validation loss increases.
- Increase the amount of training data or apply **Data Augmentation**.
- Simplify genetic programming trees to reduce computational cost.
- Use efficient optimizers such as **Adam** for faster convergence.

Conclusion

Overfitting and high computational cost in Deep Neural Networks combined with Genetic Programming can be reduced by debugging model complexity, monitoring validation performance, and applying optimization techniques such as regularization, dropout, pruning, early stopping, and efficient optimizers. These methods improve generalization while reducing computation time.

Python Program

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Load dataset
digits = load_digits()
X = digits.data
y = digits.target

# Normalize data
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Deep Neural Network (MLP)
model = MLPClassifier(
    hidden_layer_sizes=(128, 64),
    activation='relu',
    solver='adam',
    alpha=0.0001,          # L2 Regularization
    max_iter=300,
    random_state=42
)
```

```
# Train model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Test Accuracy:", accuracy)
```

Test Accuracy: 0.9722222222222222

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization. (5 Marks)

Answer

A **Decision Tree** model may achieve **high accuracy on the training dataset** but perform poorly on unseen test data. This indicates **overfitting**, where the model memorizes the training data instead of learning general patterns. Proper debugging and optimization techniques help improve the model's generalization.

Possible Causes

1. Overfitting

- The decision tree learns the training data, including noise and outliers.
- As a result, training accuracy is high, but testing accuracy is low.

2. Deep Tree Structure

- A tree with many levels creates highly specific decision rules.
- This increases model complexity and reduces generalization.

3. Noisy Data

- Incorrect, missing, or inconsistent data can create unnecessary branches.
- This negatively affects prediction performance on new data.

4. Irrelevant Features

- Features that are not useful for prediction increase the complexity of the tree.
- They may lead to incorrect splits and poor model performance.

Debugging Approaches

1. Compare training accuracy with testing accuracy.
2. Visualize the decision tree to identify unnecessary deep branches.
3. Check the maximum depth and number of leaf nodes.
4. Detect noisy or missing data during preprocessing.
5. Evaluate feature importance to identify irrelevant features.
6. Perform cross-validation to verify model stability.

Optimization Techniques

- Apply **Pre-Pruning** by limiting the maximum tree depth.
- Use **Post-Pruning (Cost Complexity Pruning)** to remove unnecessary branches.
- Perform **K-Fold Cross-Validation** to evaluate model performance.
- Apply **Feature Selection** to remove less important features.
- Clean the dataset by handling missing values and outliers.
- Tune hyperparameters such as **max_depth**, **min_samples_split**, and **min_samples_leaf**.

Conclusion

A Decision Tree may overfit because of a deep tree structure, noisy data, or irrelevant features. By applying **pruning**, **cross-validation**, **feature selection**, and **hyperparameter tuning**, the model becomes simpler, improves generalization, and achieves better performance on unseen data.

Python Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Decision Tree with pruning (max_depth)
model = DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Test Accuracy:", accuracy)
```

Test Accuracy: 1.0

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	30	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	10	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand